DETAILED PROJECT GUIDE

Intelligent Network Intrusion Detection and Attack Classification

A practical walkthrough of the UNSW-NB15 machine-learning project

> **Project in one sentence.** The application learns from labeled network-flow statistics to distinguish normal traffic from attacks, classify attack families, evaluate five models, save complete prediction pipelines, and expose them through a command-line interface and Streamlit web app.

Dataset: UNSW-NB15 partitioned training and testing CSVs

Implemented tasks: binary detection and ten-class classification

Models: Logistic Regression, Decision Tree, Random Forest, XGBoost, and MLP

Document basis: project source, generated artifacts, and real-data run results

This is an educational benchmark project, not a production-ready security control.

1. What the project does

A network intrusion detection system (IDS) monitors traffic and identifies activity that may be malicious. This project operates on *flow records*: compact summaries of network conversations, rather than raw packet payloads. A record contains measurements such as duration, byte counts, protocol, TCP state, packet rates, and connection counts.

The project learns the relationship between those features and the dataset's labels. It has two related prediction tasks:

Task	Target	Classes predicted	Question answered
Binary	`label`	Normal, Attack	Is this flow benign or malicious?
Multiclass	`attack_cat`	Normal plus nine attack families	Which known category best describes this flow?

The same raw feature columns are used in each task; only the target label changes. The output is a predicted class and, when supported by the model, class probabilities that the app displays as confidence.

End-to-end flow

```
Partitioned CSVs
    -> clean column names, category text, and infinite values
    -> separate target labels from input features
    -> fit preprocessing on training data
    -> train five candidate classifiers
    -> evaluate on held-out test data
    -> save models, metrics, reports, and plots
    -> predict from CLI or Streamlit
```

Why this is useful

Accuracy alone can hide poor performance on uncommon attacks. The project therefore reports macro-F1, per-class F1, attack detection rate, and false-alarm rate alongside accuracy and ROC-AUC. An analyst can compare models based on the cost of missed attacks versus unnecessary alerts.

2. Dataset and labels

UNSW-NB15 was created by UNSW Canberra using a cyber range to combine normal activity with synthetic attack behaviours. The project uses the supplied partitioned data, which contains the fields needed by the code, including `label` and `attack_cat`.

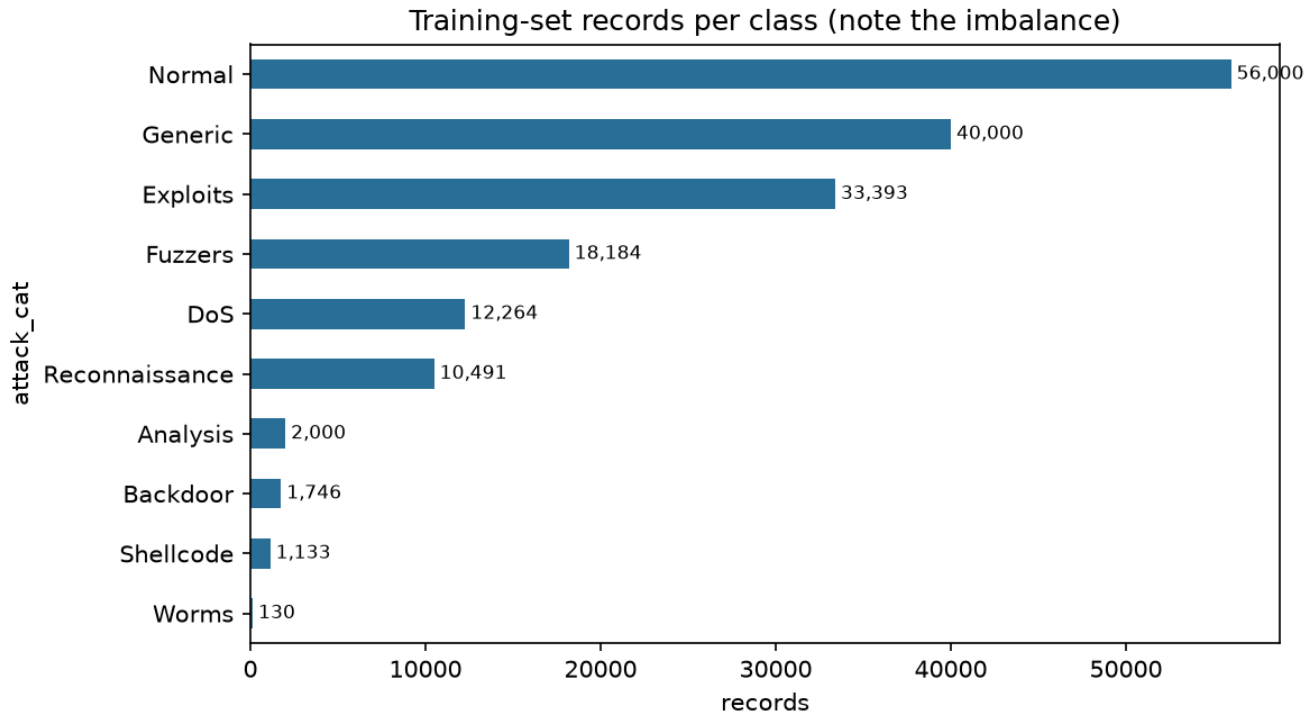
Split	Rows used	Purpose
Training	175,341	Fit preprocessing and model parameters
Testing	82,332	Final held-out evaluation and example predictions

The project data folder should contain `data/UNSW_NB15_training-set.csv` and `data/UNSW_NB15_testing-set.csv`. The training CSV has 45 columns in the current copy. After excluding ID and labels, 42 feature columns are passed to the models.

Ten multiclass labels

Label	General description	Training rows
Normal	Benign traffic	56,000
Generic	Generic attack family	40,000
Exploits	Attempts to use software vulnerabilities	33,393
Fuzzers	Probe services with malformed or random inputs	18,184
DoS	Denial-of-service activity	12,264
Reconnaissance	Scanning and information gathering	10,491
Analysis	Analysis and probing activity	2,000
Backdoor	Covert access attempts	1,746
Shellcode	Small payloads associated with code execution	1,133
Worms	Self-replicating malware activity	130

> **Class imbalance matters.** Generic has 40,000 training examples, while Worms has 130. A classifier can score well overall by recognizing common classes and still miss rare attacks. Macro-F1 and the per-class report help reveal this.



The class counts above come from the actual training run. Values describe this partition and should not be treated as the distribution of live network traffic.

3. Cleaning and preprocessing

Cleaning

`src/data_loader.py` reads a CSV and applies consistent cleaning without dropping rows: column names are lower-cased and trimmed, category text is trimmed and normalised to lower case, attack category text is trimmed, and positive/negative infinity is replaced with missing values. In the current real-data training file, the EDA summary reports zero missing and zero infinite cells before imputation.

For a training run, `id`, `label`, and `attack_cat` are excluded from the feature matrix. The selected task determines which target is returned. The optional `--dedupe` flag removes duplicate training rows (excluding ID from comparison); it does not deduplicate test records. The reported runs did not enable it, and EDA found 67,601 duplicate training records when ignoring ID.

Feature transformation

Feature type	Processing	Purpose
Numeric	Median imputation	Fill missing numeric measurements robustly
Numeric	Signed log transform: sign(x) * log(1 + abs(x))	Compress large ranges while retaining negative values
Numeric	StandardScaler	Put numeric measurements on comparable scales
`proto`, `service`, `state`	One-hot encoding	Represent categories numerically; unknown/rare values are handled

All transforms are combined with the classifier in a scikit-learn pipeline. In a normal fit, preprocessing is learned using training rows and then applied to test rows. This prevents test measurements from setting imputation, scaling, or category-encoding parameters.

Optional correlated-feature filtering

`--drop-correlated` identifies numeric columns with absolute pairwise correlation greater than 0.95 in the training data, then removes the selected columns from both splits. This experiment is optional and was not part of the completed baseline runs.

Why no payload parsing?

The inputs are engineered flow statistics. The project does not ingest packet captures, inspect packet contents, reconstruct sessions, or observe behaviour over time. Its prediction scope is limited to information represented by each supplied row.

4. Models and training strategy

Model	Role and configuration
Logistic Regression	Linear baseline; class-balanced; up to 300 iterations.
Decision Tree	Single interpretable tree; class-balanced.
Random Forest	Ensemble of 300 trees by default; parallel fitting and balanced class weights.
XGBoost	Gradient-boosted trees; 300 estimators by default; balanced sample weights are supplied by the training code.
MLP	Neural network with hidden layers of 128 and 64 units, early stopping, and a 100-iteration cap.

Imbalance options

Tree models and Logistic Regression use balanced class weights where supported. XGBoost receives balanced per-row sample weights. The optional SMOTE run transforms training features and raises classes below 10,000 training examples toward that target using nearest-neighbour interpolation. The test set remains unchanged. SMOTE is a comparison, not a guaranteed improvement.

Hyperparameter tuning

The code provides optional randomized search over model-specific parameter grids. It uses macro-F1 scoring and three-fold cross-validation on training data, and can subsample up to 40,000 rows for tuning. The final model is then fit on the training data. This tuning experiment was not run in the reported results.

Reproducibility

The project sets a shared random seed of 42 for estimators and sampling. Results can still vary with library versions, hardware, parallel execution, or changes to input data and parameters. Run configuration and measured outputs are saved alongside results where available.

5. How model quality is measured

Metric	Meaning	Interpretation
Accuracy	Correct predictions divided by all predictions	Overall correctness; can be dominated by common classes.
Precision	Correct predicted class divided by predictions of that class	How trustworthy alerts or category predictions are.
Recall	Correct predicted class divided by all true examples of that class	How much of a class the model finds.
Macro-F1	Unweighted mean of F1 over classes	Gives rare and common classes equal importance.
Weighted-F1	Class F1 averaged by true class support	Reflects observed class frequency.
Attack detection rate	True attacks predicted as any attack / all true attacks	Overall sensitivity to attacks, regardless of attack family.
False-alarm rate	Normal flows predicted as attack / all normal flows	Share of benign records incorrectly flagged.
ROC-AUC	Ranking quality across decision thresholds	Binary AUC or macro one-vs-rest AUC for multiclass.

In multiclass work, attack detection rate and false-alarm rate are calculated after grouping every non-Normal class as “attack.” They are useful operational summaries, but do not replace attack-family metrics. The detailed classification reports and per-class F1 CSVs should be examined before selecting a model.

Confusion matrix

The project plots row-normalised confusion matrices. A row corresponds to a true class; its values show the fraction assigned to each predicted class. A strong diagonal indicates correct classification. Off-diagonal cells show which attack families are confused.

Feature importance

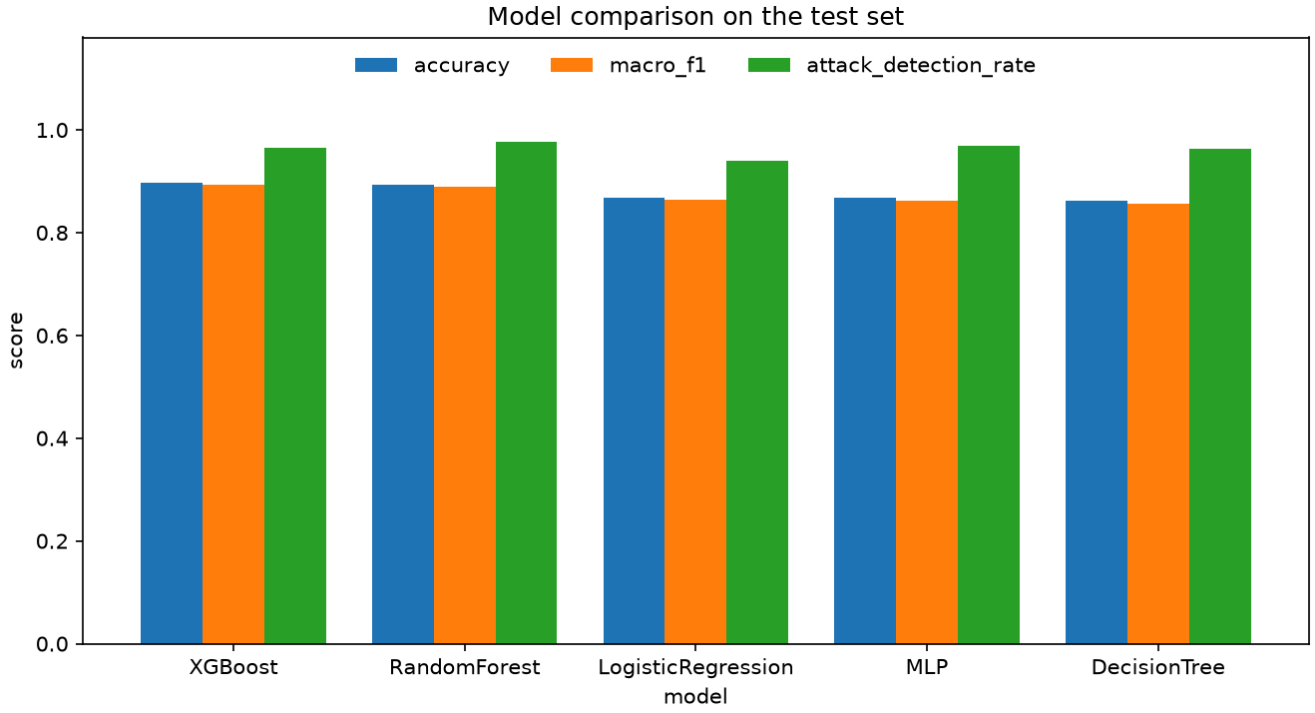
Decision Tree, Random Forest, and XGBoost expose tree-based feature importance. The project writes CSVs and top-feature plots. Importance indicates model use, not causation, and correlated inputs can share or distort importance.

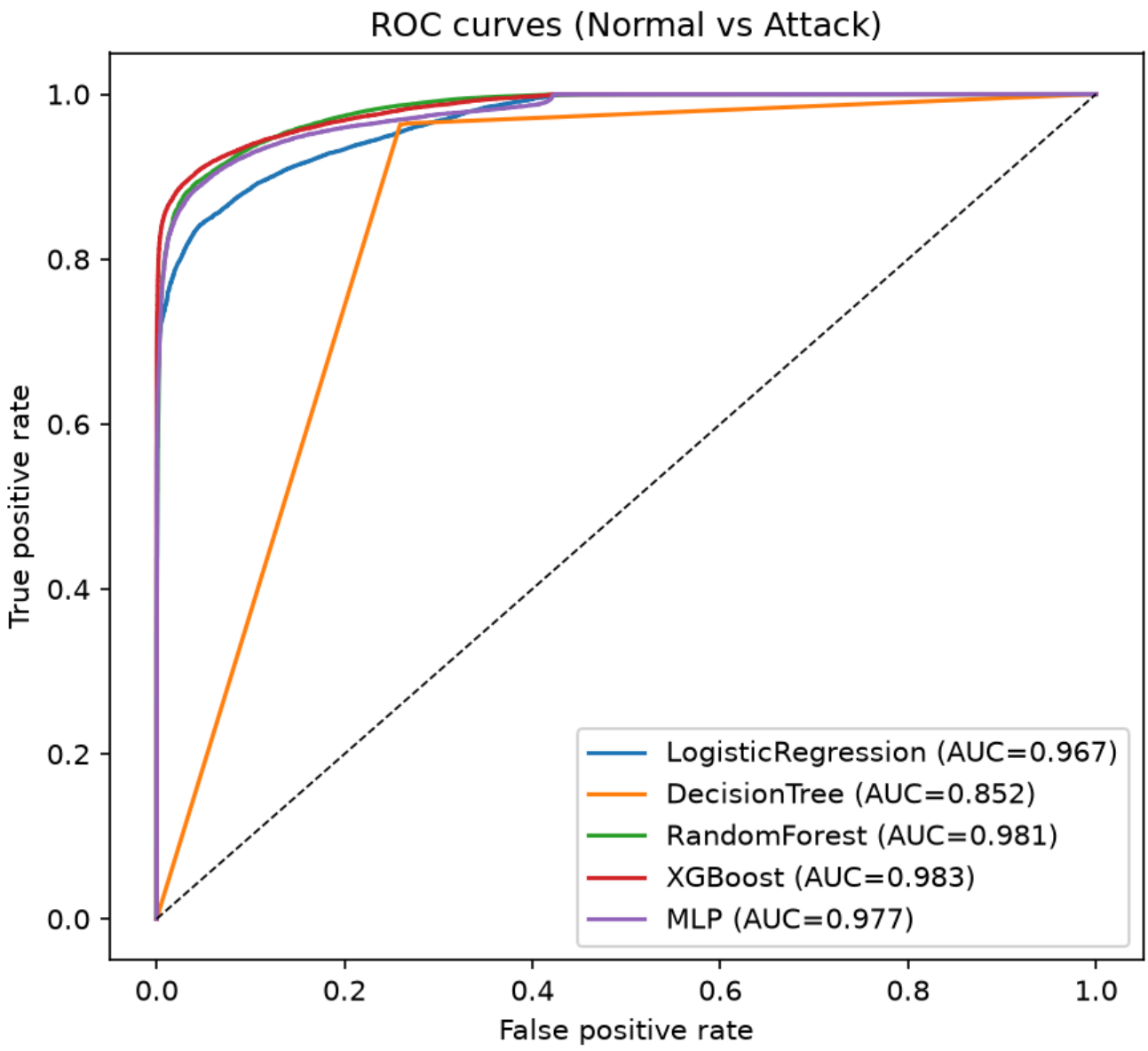
6. Results from the completed real-data run

Rows below are measured on the supplied held-out test partition. They are specific to this run and dataset; they are not guarantees for other networks.

Binary detection

Model	Accuracy	Macro-F1	Attack detection	False alarm	ROC-AUC
Logistic Regression	0.8681	0.8642	0.9407	0.2209	0.9670
Decision Tree	0.8620	0.8563	0.9636	0.2625	0.8523
Random Forest	0.8926	0.8889	**0.9766**	0.2103	0.9814
XGBoost	**0.8964**	**0.8934**	0.9661	**0.1890**	**0.9827**
MLP	0.8681	0.8627	0.9686	0.2551	0.9767

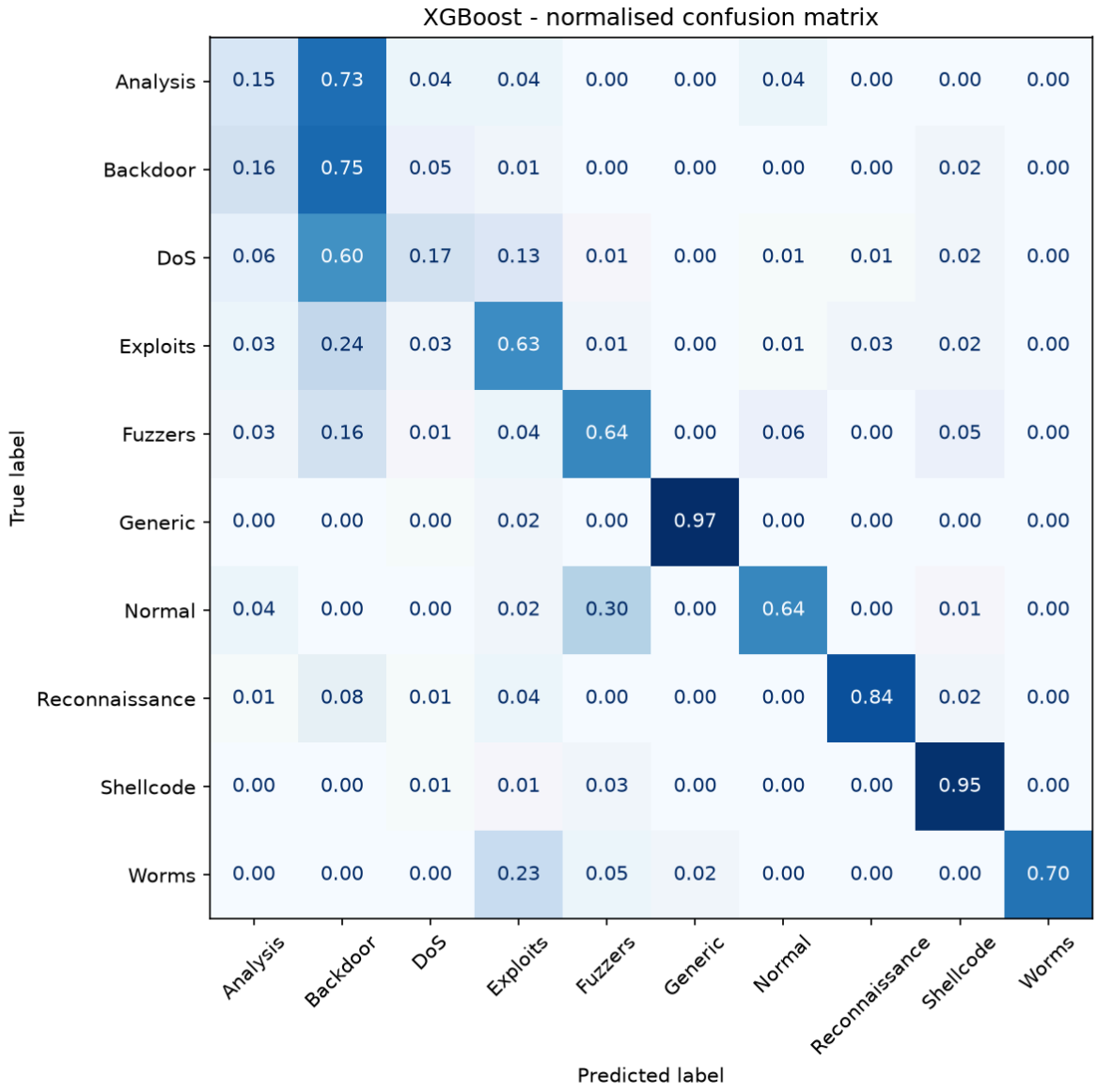


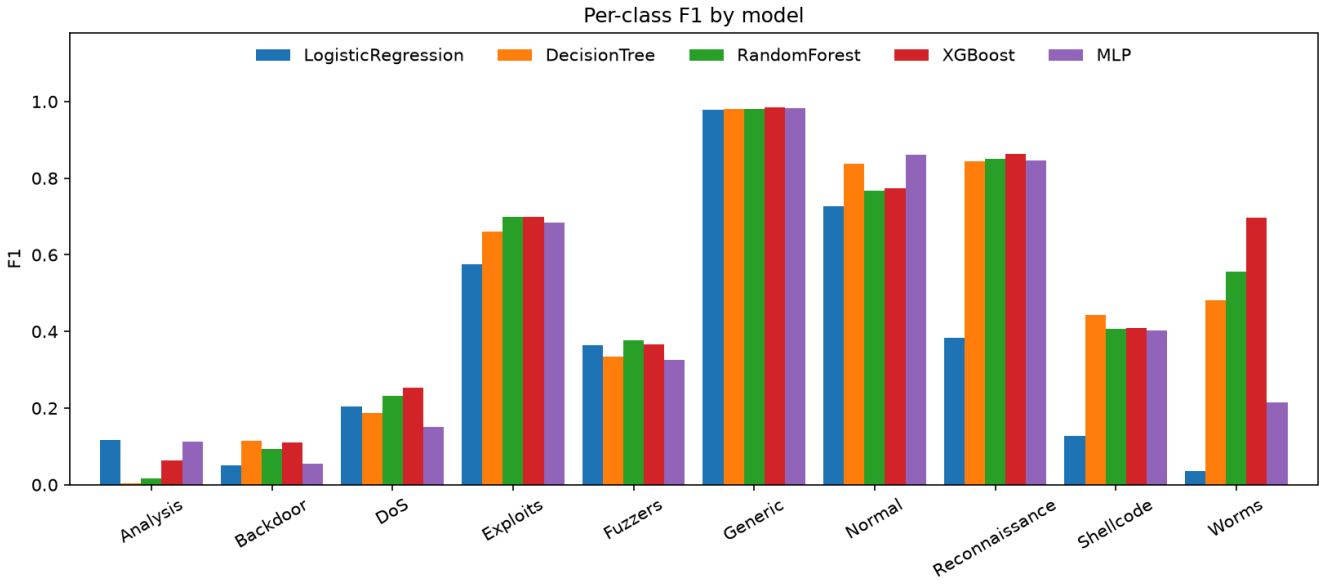


Multiclass and SMOTE summary

Experiment	Highest accuracy	Highest macro-F1	Observation
Multiclass baseline	MLP: 0.7674	XGBoost: 0.5219	Accuracy and balanced class quality rank models differently.
Multiclass + SMOTE	MLP: 0.7221	XGBoost: 0.5106	Best macro-F1 decreased from baseline.

For baseline multiclass XGBoost, Generic F1 was about 0.985 and Exploits F1 about 0.700, while Analysis F1 was about 0.063 and Backdoor F1 about 0.111. That unevenness is why the headline accuracy alone is inadequate. XGBoost binary was the strongest balanced choice across the listed aggregate metrics, while Random Forest detected the largest fraction of attacks. A deployment would need a threshold and error-cost decision based on its requirements.


XGBoost - normalised confusion matrix
True label
Predicted label
Analysis
Backdoor
DoS
Exploits
Fuzzers
Generic
Normal
Reconnaissance
Shellcode
Worms
0.15 0.73 0.04 0.04 0.00 0.00 0.04 0.00 0.00 0.00
0.16 0.75 0.05 0.01 0.00 0.00 0.00 0.00 0.02 0.00
0.06 0.60 0.17 0.13 0.01 0.00 0.01 0.01 0.02 0.00
0.03 0.24 0.03 0.63 0.01 0.00 0.01 0.03 0.02 0.00
0.03 0.16 0.01 0.04 0.64 0.00 0.06 0.00 0.05 0.00
0.00 0.00 0.00 0.02 0.00 0.97 0.00 0.00 0.00 0.00
0.04 0.00 0.00 0.02 0.30 0.00 0.64 0.00 0.01 0.00
0.01 0.08 0.01 0.04 0.00 0.00 0.00 0.84 0.02 0.00
0.00 0.00 0.01 0.01 0.03 0.00 0.00 0.00 0.95 0.00
0.00 0.00 0.00 0.23 0.05 0.02 0.00 0.00 0.00 0.70


Per-class F1 by model
LogisticRegression
DecisionTree
RandomForest
XGBoost
MLP
F1
0.0
0.2
0.4
0.6
0.8
1.0
Analysis
Backdoor
DoS
Exploits
Fuzzers
Generic
Normal
Reconnaissance
Shellcode
Worms

7. What SMOTE changed

SMOTE increased the number of examples used to train minority classes by synthesising feature vectors from nearby examples. It does not add newly observed attacks; it changes the training distribution to give rare classes more influence.

XGBoost class	Baseline F1	With SMOTE F1	Change
DoS	0.253	0.387	Improved in this run
Shellcode	0.409	0.370	Decreased
Worms	0.697	0.552	Decreased; few real examples available
Overall macro-F1	0.5219	0.5106	Decreased

These outcomes show that resampling may help one class and hurt another. The right choice depends on per-class recall/precision and the impact of false negatives and false positives. Synthetic samples can only interpolate patterns present in the minority training examples; they cannot represent entirely new attack behaviour.

Important warning: benchmark versus operations

The test split is a fixed benchmark partition. A good score here does not show that the model will generalise to traffic collected at another time, network, device population, or attack campaign. A real deployment needs external validation, monitoring for drift, analyst review, and a safe alerting policy.

8. Run the project and use its outputs

Install and test

```
python -m venv .venv
# Windows PowerShell:
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
pytest -q
```

Run core experiments from the repository root

```
python -m src.eda
python -m src.train --task binary --tag real
python -m src.train --task multiclass --tag real
python -m src.train --task multiclass --smote --tag real_smote
```

The completed artifacts use the suffix `real` (or `real_smote`) so they do not overwrite untagged runs. The standard all-steps script is `run_all.sh` and expects a Bash environment.

Batch predictions with the saved multiclass pipeline

```
python -m src.predict ^
  --model models/multiclass_real/best_model.joblib ^
  --input data/UNSW_NB15_testing-set.csv ^
  --output predictions.csv
```

The Windows line-continuation caret above is for Command Prompt. In PowerShell, put the command on one line or use PowerShell's backtick continuation. The prediction command writes the input rows plus predicted class, attack flag, and confidence columns (when the classifier supports probabilities). The saved pipeline handles feature alignment and preprocessing.

Launch the web application

```
streamlit run app/streamlit_app.py
```

In the sidebar select a task/run folder and model artifact. Upload a CSV with the expected UNSW-NB15 feature columns, or use the test-set sample button. The page reports predicted class counts, attack share, mean confidence, shows the most confident flagged flows, permits single-row probability inspection, and provides a predictions CSV download. When the uploaded CSV includes target labels, the app also shows accuracy and macro-F1 for those rows.

9. Project structure and saved artifacts

Path	Purpose
`src/config.py`	Dataset paths, label names, output locations, and random seed.
`src/data_loader.py`	CSV loading, normalisation, cleaning, and task target separation.
`src/preprocessing.py`	Numeric transformations, categorical encoding, and optional correlation filter.
`src/models.py`	Five estimators and optional hyperparameter grids.
`src/train.py`	Training, optional tuning/SMOTE, evaluation, plotting, and artifact save.
`src/evaluate.py`	Metrics, classification report, confusion matrix and comparison charts.
`src/predict.py`	Load saved artifacts and predict from a CSV or dataframe.
`src/eda.py`	Dataset overview, class distribution, feature plots, and summary tables.
`app/streamlit_app.py`	Interactive browser interface for selecting models and reviewing predictions.
`tests/test_pipeline.py`	Fast synthetic-data tests for cleaning, preprocessing, training, and prediction.

Where to find run products

- `models/binary_real/`, `models/multiclass_real/`, `models/multiclass_real_smote/`: each model and `best_model.joblib`.
- `reports/results/<run>/metrics_summary.csv`: aggregate metrics, train time, and inference time.
- `reports/results/<run>/per_class_f1.csv` and `report_*.txt`: per-class outcomes.
- `reports/figures/<run>/`: confusion matrices, model comparison, ROC and feature-importance plots.
- `reports/results/eda/` and `reports/figures/eda/`: exploratory-analysis tables and figures.

> **Large model file.** The SMOTE Random Forest artifact is approximately 1.5 GB because of the fitted forest and oversampled training. Keep enough disk space before copying or backing up the complete model directory.

10. Limitations and next steps

Known limitations

- The dataset is lab-generated and dates from 2015; modern operational traffic can differ.
- Flow-level statistics omit packet payloads and longer-term host/session context.
- Rare attack categories have few examples and unstable estimates; Worms has 130 training records in this split.
- The model is closed-set: it selects among known classes and is not an unknown-attack detector.
- Logistic Regression reached its configured 300-iteration limit in multiclass runs. Its warning and results are documented; tuning iterations/solver is a reasonable follow-up.
- The test partition is used for reporting, not model selection. Further model choices should be based on a validation split or training-only cross-validation, not repeated test-set tuning.

Recommended follow-up

1. Run the optional Random Forest/XGBoost tuning experiment and reduced-feature experiment, each under a distinct tag.
2. Review class-level confusion matrices and choose operating thresholds with a stated cost of missed attacks versus false alarms.
3. Validate on a separate, more recent dataset or locally collected data before any operational use.
4. Consider probability calibration, drift monitoring, analyst feedback, and explicit rejection/unknown-class handling.
5. Check memory/storage requirements before loading the SMOTE Random Forest artifact.

Completion and validation status

The core end-to-end workflow has been run on the real partitioned dataset: EDA, binary training, multiclass training, and multiclass with SMOTE. The automated suite passes four tests. The saved multiclass model produced predictions and ten-class probability vectors for the entire 82,332-row test CSV; the Streamlit health endpoint also responded during verification. Hyperparameter tuning and correlated-feature removal remain optional experiments, not requirements for the completed baseline pipeline.

Dataset source: UNSW Canberra, UNSW-NB15. Cite N. Moustafa and J. Slay, “UNSW-NB15: a comprehensive data set for network intrusion detection systems,” MilCIS, 2015, when using the dataset.